

PROMPT ENGINEERING

ZERO SHOT PROMPTING (ROLE BASED):

PROMPT (WITHOUT ROLE): Write a cold email to a potential client about our analytics software.

Hi [Client Name],

I hope you're doing well.

My name is [Your Name], and I'm reaching out from [Company Name]. We help businesses make smarter, data-driven decisions through our analytics software.

Our platform enables organizations to:

- Track key performance metrics in real time
- Generate actionable insights from business data
- Create customizable dashboards and reports
- Improve operational efficiency and decision-making

Many companies struggle with scattered data and time-consuming reporting processes. Our solution simplifies analytics by bringing all your critical information into one easy-to-use platform.

I'd love to learn more about your current analytics process and discuss how our software could help your team save time and uncover valuable insights.

Would you be available for a brief 15-minute call next week?

Thank you for your time, and I look forward to hearing from you.

Best regards,

[Your Name]

[Job Title]

[Company Name]

[Phone Number]

[Email Address]

[Website]

PROMPT (WITH ROLE): You are a senior sales rep who has closed 200+ enterprise deals. You know that cold emails fail when they talk only about features. They work when they name a specific pain point, Write a cold email to the VP of Marketing about our analytics software.

Hi [First Name],

Most marketing teams I speak with have no shortage of data.

The challenge is that campaign performance, attribution, and revenue metrics often live in different systems, making it difficult to answer a simple question:

"Which marketing investments are actually driving growth?"

As a result, teams spend hours building reports, defending budgets, and making decisions based on incomplete information.

We built [Product Name] to solve that problem.

Our analytics platform brings marketing, sales, and revenue data into a single view, helping marketing leaders:

- Identify which channels generate the highest ROI
- Track campaign performance in real time
- Improve attribution accuracy
- Make faster, data-backed budget decisions

I'd be interested to learn how your team currently measures marketing effectiveness and whether visibility across your data sources is a challenge today.

Would you be open to a brief 15-minute conversation next week?

Best regards,

[Your Name]

[Title]

[Company]

[Phone Number]

FEW SHOT PROMPTING (STRUCTURED):

PROMPT: Product Launch (PulseX Band)

Subject: Your wrist just got smarter.

Look, we'll skip the part where we pretend you've been waiting for this. You probably haven't. But you should have been.

PulseX Band tracks your heart rate, steps, and sleep — and it does it without looking like a medical device from 2015. The display is sharp. The strap doesn't collect sweat like a sponge. And setup takes about 90 seconds, which is less time than you spent reading your last WhatsApp group argument.

Starts at ₹4,999. No subscription. No nonsense. No "premium tier" that locks away half the features.

Just put it on and go.

[Shop PulseX Band →]

Subject: Your wrist just got smarter.

Look, we'll skip the part where we pretend you've been waiting for this. You probably haven't.

But if you've ever wondered whether you're sleeping enough, moving enough, or doing enough to stay healthy, you should have been.

PulseX Band tracks your heart rate, steps, and sleep — and it does it without looking like a medical device from 2015.

The display is sharp. The strap stays comfortable all day. Setup takes about 90 seconds, which is probably less time than you spent deciding what to watch last night.

And unlike some wearables, there are no subscriptions, hidden upgrades, or features locked behind a "premium" plan.

Starts at ₹4,999.

Just put it on and get moving.

[Shop PulseX Band →]

PROMPT : Feature Update (Sleep Tracking)

Subject: We watched you sleep. In a cool way.

PulseX now tracks your sleep stages — light, deep, REM — so you can finally stop guessing why you feel like garbage at 2pm.

Here's how it works: wear your band to bed. Wake up. Open the app. See your sleep score. Judge yourself accordingly. It also tracks how often you woke up, how long it took you to fall asleep, and whether your "8 hours" was actually 5 hours of tossing and 3 hours of real rest.

No setup needed. The update's already live. Just open the app and tap Sleep.

Fair warning — the data might hurt your feelings.

[Check your sleep score →]

Hi [First Name],

Good news: PulseX now tracks your sleep stages.

Slightly less good news: you might discover you've been sleeping a lot worse than you thought.

PulseX now tracks light, deep, and REM sleep, so you can finally stop guessing why you feel exhausted at 2 PM despite getting a "full night's sleep."

Here's how it works:

Wear your band to bed.

Wake up.

Open the app.

See your sleep score.

Accept reality.

You'll also see:

- How often you woke up during the night
- How long it took you to fall asleep
- Whether your "8 hours" was actually 5 hours of sleep and 3 hours of negotiating with your pillow

The best part? There's nothing to set up.

The update is already live.

Just open the app and tap **Sleep**.

Fair warning: the data may hurt your feelings.

[Check your sleep score →]

— Team PulseX

PROMPT : Sale Announcement (End of Season)

Subject: 30% off. No catch. Okay, one catch.

The catch is it ends Sunday.

Everything in the store — bands, chargers, straps, the works — 30% off. We'd say "while stocks last" but honestly we have plenty. We just want you to feel urgency. Is it working?

If you've been thinking about upgrading your strap or grabbing a second charger for the office, this is the moment. If you haven't been thinking about it, maybe start. We're not going to beg. But we're also not running this sale again until we feel like it.

Code: PULSE30 at checkout. Ends Sunday 11:59pm.

[Shop the sale →]

Hi [First Name],

We've got good news and bad news.

The good news: everything in the store is 30% off.

The bad news: it ends Sunday.

Bands. Chargers. Straps. The whole lineup.

We'd usually say "while stocks last," but honestly, we have plenty. We just wanted to add a little drama. Did it work?

If you've been meaning to upgrade your strap, grab a spare charger for work, or finally replace the accessory that's somehow survived three years of abuse, now's the time.

And if you haven't been thinking about it...

Maybe start.

Use code **PULSE30** at checkout and take **30% off everything**.

Offer ends Sunday at 11:59 PM.

After that, prices go back to normal and we'll all pretend this never happened.

[Shop the Sale →]

— Team PulseX

CRAFT PROMPT :

<Context> I am currently designing a production grade pipeline using kafka + DBT + Snowflake and terraform integration. My team is full of high quality engineers, and hence I want to make sure, that the code scripted by me too stands out and abides by all the software engineering principles. I want this code to be scalable and optimized in the best possible way.

Role: Act as Senior software engineer with 2 decades of experience, who has worked on high quality enterprise-level scalable applications and is adept with data engineering knowledge and impeccable programming skills. You have worked on a variety of streams like data engineering, machine learning, database engineering. You have unique way for explaining complex concepts in a simpler manner, and also reviewing code in the best possible manner and suggesting the most potent additions to the code which enhances the application quality.

<Action>: I want you to review the code and suggest me where I the code be engineered in a better way, where can it be optimized, which part of the code seems unnecessary and where can i improve better. Which are the strengths and weaknesses in the code, and also suggest me the write versions of the code, after suggesting the changes.

<Format: > OUtput should be strictly in markdown format. Only if required, perform comparative analysis in tabular format.

Constraints: The code review should be strictly with the code uploaded. Avoid explaining too technically, rather it should be simpler to follow and implement. Be precise and verbose at the same time.

1. Architecture Review

I'll assess:

- Scalability
- Fault tolerance
- Idempotency
- Maintainability
- Cost optimization
- Security
- Observability
- Disaster recovery readiness

Example questions:

- Are Kafka consumers safe against duplicate events?
- Is Snowflake compute usage optimized?

- Are dbt models following proper layer separation (Bronze/Silver/Gold)?
 - Is Terraform state management enterprise-ready?
 - Are secrets managed securely?
-

2. Code Quality Review

I'll identify:

Strengths

Example:

- ✓ Clear module separation
- ✓ Good variable naming
- ✓ Proper use of dbt sources
- ✓ Reusable Terraform modules

Weaknesses

Example:

- ✗ Hardcoded environment values
 - ✗ Repeated SQL logic
 - ✗ Missing exception handling
 - ✗ Tight coupling between Kafka and warehouse logic
-

3. Optimization Opportunities

I will specifically highlight:

Area	Current Issue	Recommended Improvement	Expected Benefit
Kafka	Example issue	Better approach	Impact
Snowflake	Example issue	Better approach	Cost/performance
dbt	Example issue	Better approach	Maintainability
Terraform	Example issue	Better approach	Scalability

4. Production Readiness Assessment

I'll check for:

Kafka

- Consumer groups
- Retry strategy
- Dead letter queue (DLQ)
- Offset management
- Idempotent writes

Snowflake

- Clustering strategy
- Warehouse sizing
- Query optimization
- Incremental loading

dbt

- Source freshness tests
- Schema tests
- Generic tests
- Documentation coverage
- Incremental models

Terraform

- Remote state
- State locking
- Environment separation
- Reusable modules
- Variable validation

5. Software Engineering Principles

I'll evaluate adherence to:

SOLID

- Single Responsibility
- Open/Closed
- Dependency Inversion

Clean Code

- Readability
- Naming
- Modularity

DRY

Detection of duplicated logic.

KISS

Identification of unnecessary complexity.

6. Enterprise-Level Enhancements

I'll suggest improvements such as:

Observability

OpenTelemetry

Prometheus

Grafana

Snowflake query monitoring

dbt artifacts monitoring

Data Quality

dbt tests

Great Expectations

Monte Carlo

Custom anomaly detection

Security

RBAC

Snowflake Roles

Secrets Manager

Least Privilege Access

Encryption

CI/CD

Terraform plan validation

dbt build pipelines

Automated testing

Promotion across environments

7. Refactored Version

For every issue found, I'll provide:

Current Code

Existing implementation

Problem

Why this becomes difficult to maintain or scale.

Improved Version

Refactored implementation

Benefit

Why the new implementation is better.

8. Senior Engineer Verdict

At the end, I'll provide:

Architecture Score: X/10

Scalability: X/10

Maintainability: X/10

Performance: X/10

Reliability: X/10

Security: X/10

Observability: X/10

Top 5 Improvements:

- 1.
- 2.
- 3.
- 4.
- 5.

Production Readiness:

READY / NEEDS IMPROVEMENT / NOT READY

MEETING TRANSCRIPT :

Weekly AI Pipeline Sync — Document Intelligence Platform

Date

Thursday, March 12, 2026 — 10:30 AM IST

Duration

47 minutes

Location

Google Meet (recorded)

Attendees

Priya Sharma (VP of Product), Arjun Mehta (ML Engineering Lead), Sneha Iyer (Data Engineering Manager), Rohit Desai (Senior ML Engineer), Kavya Nair (Product Manager)

Absent

Nikhil Rao (CTO) — at board meeting

TRANSCRIPT BEGINS

[00:00]

Priya Sharma: Alright, let's get started. Nikhil's at the board meeting so he won't be joining, but he wants an update afterward, especially on the timeline. So let's make sure we're clear on where things stand. Arjun, do you want to kick off with the model update?

[00:32]

Arjun Mehta: Sure. So the good news is the extraction model is performing significantly better after the retraining last week. We switched from the fine-tuned GPT-3.5 to a Mistral 7B base with LoRA adapters, and the field-level accuracy on invoices went from 78% to 91%. That's on our internal test set of 2,200 documents.

Priya Sharma: That's a big jump. Is that across all document types or just invoices?

Arjun Mehta: Just invoices for now. Purchase orders are at about 84%, which is up from 72%, so also a solid improvement. But contracts are still a problem. We're hovering around 67% on the contract extraction, and honestly I'm not sure fine-tuning alone is going to get us where we need to be on those.

[01:45]

Rohit Desai: Yeah, the contract issue is structural. Contracts have way more variability in layout — different firms, different jurisdictions, different formatting conventions. We've been talking about a two-stage approach where we first classify the contract type and then route it to a specialized extraction head, but that adds a lot of complexity to the pipeline.

Priya Sharma: How much complexity are we talking? Another two weeks? A month?

Rohit Desai: Honestly, if we want to do it properly, probably three to four weeks of engineering time. And that's just the model work — Sneha's team would need to build out the routing logic on the data pipeline side too.

[02:48]

Sneha Iyer: Hold on, let me push back on that a little. The routing logic isn't the hard part. We can have that up in a few days, it's basically a classifier output feeding into an Airflow branch operator. The hard part is the training data. Do we even have enough labeled contracts to train specialized extraction heads? Last I checked we had maybe 400 labeled contracts total, and that's across all types.

Arjun Mehta: We have 430, to be exact. And if we split by type — NDAs, service agreements, licensing agreements — the smallest bucket is licensing with only 67 examples. That's not enough to fine-tune anything reliable.

Priya Sharma: So what's the alternative? We can't ship with 67% accuracy on contracts. That's the one document type our enterprise clients actually care about the most.

[04:12]

Arjun Mehta: I think the realistic options are: one, we invest in a labeling sprint and get the training data up to at least 200 per contract type before we attempt the two-stage approach. Two, we use a prompted approach for contracts instead of fine-tuning — basically long-context extraction with Claude or GPT-4 using carefully engineered prompts. Three, we descope contracts from the V1 launch and add them in V1.1.

Kavya Nair: Option three is not going to fly with the sales team. I've already got two enterprise prospects — Reliance Digital and Tata Communications — who specifically asked about contract extraction in their last calls. If we pull that from V1, we might lose both deals.

Priya Sharma: Agreed, descopeing is a last resort. What about option two — the prompted approach? Arjun, is that viable?

[05:30]

Arjun Mehta: It's viable but it changes our cost structure. Right now the fine-tuned Mistral model runs on our own GPU cluster — inference cost is basically zero after the initial hardware investment. If we switch contracts to a prompted approach with an external API, we're looking at roughly 12 to 15 paisa per page, which adds up fast for contract-heavy clients.

Rohit Desai: It's also slower. Fine-tuned model does extraction in about 1.2 seconds per page. A prompted approach with Claude is more like 6 to 8 seconds depending on contract length. For a 40-page contract, that's over five minutes.

Kavya Nair: Is five minutes a dealbreaker though? These aren't real-time use cases. A client uploads a contract and gets the extraction back in five minutes — that's still way faster than their current process, which is someone manually reading the thing for an hour.

[06:52]

Sneha Iyer: I'm less worried about the latency and more worried about the reliability. If we're calling an external API for a core feature, we need proper retry logic, fallback handling, rate limiting — all of that. And we need to figure out the data residency question. Some of these enterprise clients have strict requirements about where their data gets processed. Are we okay sending their contracts to Anthropic's API?

Priya Sharma: That's a good point. Kavya, do we know the data residency requirements for Reliance and Tata?

Kavya Nair: Reliance definitely has a data residency clause — everything has to stay within India. I'd need to check the specifics for Tata. Let me follow up with their procurement team and get clarity this week.

Priya Sharma: Okay, please do that by Monday. We can't make a technical decision without knowing the constraints.

[08:15]

Priya Sharma: Alright, here's what I'm thinking. Let's pursue option one and two in parallel. Arjun, can you and Rohit start the labeling sprint for contracts? Get the annotation team to prioritize contract labeling — I want at least 200 labeled examples per contract type within two weeks.

Arjun Mehta: That's aggressive. The annotation team is currently finishing the invoice validation batch. We'd need to pull them off that.

Priya Sharma: The invoice model is already at 91%. We can pause the invoice annotations. Contracts are the bottleneck.

Arjun Mehta: Fair enough. I'll talk to Meera on the annotation team today and get them redirected.

[09:35]

Priya Sharma: Good. And simultaneously, Rohit, can you prototype the prompted approach? I want to see what accuracy we can get with a well-engineered prompt on the 430 contracts we already have. Give me a benchmark by end of next week.

Rohit Desai: I can do the benchmark, but end of next week is tight. I'm also supposed to be fixing the table extraction bug that's been blocking the QA team.

Sneha Iyer: That table extraction bug is actually causing issues on my side too. The downstream ETL pipeline breaks every time it encounters a document with malformed table outputs. We've had 14 pipeline failures this week alone because of it.

Priya Sharma: Okay, how severe is the table bug? Is it a 'things crash' problem or a 'things are slightly wrong' problem?

Sneha Iyer: Things crash. The pipeline fails, the document gets sent to a dead letter queue, and we have to manually reprocess. It's costing us about two hours of engineering time per day just babysitting it.

[11:20]

Priya Sharma: That's not acceptable. Rohit, fix the table bug first. That's priority one. Then start the contract prompt benchmark. Can you have the bug fixed by Monday?

Rohit Desai: I think so, yeah. I've already identified the root cause — it's a parsing issue with nested tables. The model outputs valid JSON but the parser doesn't handle the nesting correctly. Should be a day of work, maybe two.

Priya Sharma: Good. Fix it by Monday, then start the contract benchmark. Get me results by the Friday after — March 27th.

Rohit Desai: Got it.

[12:30]

Kavya Nair: One more thing on the timeline. Our V1 target launch date is April 18th. That's five weeks from now. Are we still tracking for that? Because if the contract approach takes four more weeks of engineering after the benchmark results come in, we're already past the launch date.

Arjun Mehta: Honestly? I think April 18th is at risk. Even if the prompted approach works, integrating it properly with fallback handling, monitoring, and cost tracking is at least two weeks of work. And that's after we know which approach we're going with.

Priya Sharma: I hear you, but I'm not ready to move the date yet. Let's see where we are after the benchmark results on March 27th. If we need to adjust, we'll adjust then. But I don't want us to preemptively relax the timeline. Sometimes deadlines make magic happen.

Kavya Nair: Fair, but can we at least flag it to Nikhil? He's going to ask.

Priya Sharma: I'll talk to Nikhil. Let me frame it properly — we're on track for invoices and purchase orders, contracts are the open question, and we'll have clarity by end of month.

[14:45]

Priya Sharma: Okay, let's move to the data pipeline update. Sneha, where are we on the ingestion service?

Sneha Iyer: Good progress, actually. The async ingestion pipeline is fully built and deployed to staging. We're processing about 1,200 documents per hour in testing, which is well above our target of 800. The S3-to-processing queue is stable, Airflow orchestration is clean, and we've got proper dead letter queues for failed documents.

Sneha Iyer: The one outstanding issue — apart from the table bug that Rohit mentioned — is the OCR pre-processing for scanned documents. We're using Tesseract right now, and the quality is... not great on low-resolution scans. About 22% of our test documents are scanned, and for those, the extraction accuracy drops by 15 to 20 points because the OCR introduces errors before the model even sees the text.

[16:30]

Arjun Mehta: Have you looked at Azure Document Intelligence? Their OCR is significantly better than Tesseract, especially on degraded scans. We used it at my last company and it was night and day.

Sneha Iyer: I've looked at it. The accuracy is better, no question. But it's another external API dependency, another cost line, and another data residency conversation.

Kavya Nair: Also, sorry to keep bringing this up, but Azure has an India region, right? That might actually solve the data residency issue for both the OCR and the contract extraction if we go with a prompted approach through Azure OpenAI.

Arjun Mehta: Azure OpenAI doesn't have the latest Claude models though. If we're going prompted, Claude is performing noticeably better than GPT-4 on our extraction benchmarks. Like, 6 to 8 points better on contracts specifically.

[18:10]

Priya Sharma: Okay, we're going in circles a bit. Let me try to untangle this. Sneha, can you run a quick comparison of Tesseract versus Azure Document Intelligence this week? Use 50 of the worst scanned documents. I just need to know if the accuracy improvement justifies adding another vendor dependency.

Sneha Iyer: Yeah, I can have that done by Wednesday.

Priya Sharma: Great. And the data residency question — Kavya is going to find out the requirements from Reliance and Tata by Monday. Once we have that, a lot of these API decisions become clearer. Let's not go back and forth on Azure versus Anthropic until we actually know what's allowed.

Kavya Nair: Makes sense.

[19:45]

Priya Sharma: Anything else before we wrap? We're almost at time.

Rohit Desai: One quick thing — the GPU cluster is at 87% utilization. When we scale the annotation team and start running more training jobs for the contract models, we're going to hit capacity. Should I file a request for additional GPU allocation now, or wait?

Arjun Mehta: File it now. The procurement process takes two weeks minimum. Better to have it in the queue and cancel than to need it and not have it.

Priya Sharma: Agreed. Rohit, submit the GPU request today. Estimate what we'd need for the contract training runs and add a 30% buffer.

Rohit Desai: Will do.

[20:55]

Kavya Nair: Oh wait, one more thing. The product demo for the Reliance team is scheduled for April 3rd. That's three weeks out. What are we showing them? Because if contract extraction isn't ready, we need to frame the demo carefully.

Priya Sharma: Good catch. Let's show invoices and purchase orders — those are solid. For contracts, we can show a "coming soon" preview or a limited demo with NDAs only, since those are our strongest contract subtype. Kavya, can you draft a demo flow by end of next week and share it with the team?

Kavya Nair: Sure, I'll put together a demo script and share it by Friday the 20th.

[22:10]

Priya Sharma: Alright. Let me do a quick recap. Arjun — redirect the annotation team to contract labeling, target 200 per type in two weeks. Rohit — fix the table extraction bug by Monday, then start the prompted contract benchmark, results by March 27th. Also submit the GPU request today. Sneha — run the Tesseract versus Azure OCR comparison by Wednesday. Kavya — get data residency requirements from Reliance and Tata by Monday, and draft the demo flow by the 20th. I'll update Nikhil on the timeline situation. Everyone clear?

Arjun Mehta: Clear.

Sneha Iyer: Got it.

Rohit Desai: Yep.

Kavya Nair: All good.

Priya Sharma: Great. Good progress on the model side — 91% on invoices is a real milestone. Let's keep that momentum going. Talk next week.

RESPONSE:

Executive Summary

The team has made strong progress on invoice and purchase-order extraction, with invoice accuracy improving from 78% to 91% after moving from GPT-3.5 fine-tuning to a Mistral 7B + LoRA approach. However, contract extraction remains the primary project risk, currently at approximately 67% accuracy, which is below enterprise expectations. Additionally, a critical table-parsing bug is causing pipeline failures and consuming engineering time. The April 18 V1 launch date is at risk if contract extraction challenges are not resolved quickly.

Key Achievements

Model Improvements

- Invoice extraction accuracy improved from **78% → 91%**.
- Purchase order extraction improved from **72% → 84%**.
- Mistral 7B + LoRA retraining has produced significant gains.

Data Pipeline Progress

- Async ingestion pipeline successfully deployed to staging.
- Processing throughput reached **1,200 documents/hour**, exceeding the target of **800 documents/hour**.
- S3 ingestion, Airflow orchestration, and DLQ handling are functioning well.

Team Decision-Making

- The team identified contract extraction as the highest-priority bottleneck.
- A dual-track strategy was approved:
 1. Increase labeled contract data.
 2. Benchmark prompt-based extraction simultaneously.

Major Risks

1. Contract Extraction Accuracy

Current State

- Contract extraction accuracy is approximately **67%**.
- Enterprise customers care more about contracts than invoices.

Root Causes

- High document variability.
- Insufficient training data.
- Licensing agreements have only **67 labeled examples**.

Business Impact

- Potential loss of enterprise deals.
- Reliance Digital and Tata Communications explicitly requested contract extraction capabilities.

Recommended Action

- Continue the labeling sprint.
 - Target minimum **200 labeled samples per contract type**.
 - Benchmark prompt-based extraction in parallel.
-

2. Critical Table Extraction Bug

Current State

- Pipeline experienced **14 failures in one week**.
- Manual intervention required.
- Consumes roughly **2 hours of engineering time daily**.

Root Cause

- Nested table structures generate valid JSON.
- Parser cannot correctly process nested objects.

Severity

High (P1)

This is not a quality issue.

This is a reliability issue because the entire pipeline crashes.

Recommendation

- Fix immediately before any contract benchmark work.
- Add parser validation tests.
- Add malformed-table integration tests.

- Improve DLQ diagnostics.
-

3. Launch Timeline Risk

Current State

- Target launch: **April 18**
- Contract solution still undecided.
- Additional integration work may require multiple weeks.

Assessment

Area	Status
------	--------

Invoices	Green
----------	-------

Purchase Orders	Green
-----------------	-------

Contracts	Red
-----------	-----

OCR	Yellow
-----	--------

Infrastructure	Green
----------------	-------

Risk Level

Medium to High

The launch date is dependent on contract extraction progress.

Technical Decisions Under Evaluation

Option 1: Specialized Contract Models

Advantages

- Lower inference cost.
- Faster response times.
- Full control over infrastructure.

Disadvantages

- Requires significantly more labeled data.
- Additional engineering complexity.
- Estimated 3–4 weeks of implementation effort.

Option 2: Prompt-Based Extraction

Advantages

- Faster path to production.
- No need to wait for large-scale labeling.

Disadvantages

- Higher inference costs.
- Slower processing.
- Requires retries, fallback logic, rate limiting, and compliance review.

Assessment

This appears to be the most realistic short-term solution while the specialized models mature.

Infrastructure Concerns

OCR Quality

Current Situation

- Tesseract performs poorly on degraded scans.
- 22% of documents are scanned.
- OCR errors reduce extraction accuracy by 15–20 percentage points.

Recommendation

Proceed with the Azure Document Intelligence benchmark.

This experiment has high potential impact because OCR quality affects every downstream stage.

GPU Capacity

Current State

- GPU cluster utilization: **87%**.

Risk

Contract-model training may exceed available capacity.

Recommendation

Submitting the GPU request immediately is the correct decision.
Maintain the proposed 30% capacity buffer.

Action Items

Owner	Action	Deadline
Arjun	Redirect annotation team to contract labeling	Immediate
Arjun	Reach 200 labeled examples per contract type	2 weeks
Rohit	Fix table extraction bug	Monday
Rohit	Run contract prompt benchmark	March 27
Rohit	Submit GPU allocation request	Immediate
Sneha	Compare Tesseract vs Azure OCR	Wednesday
Kavya	Gather data residency requirements	Monday
Kavya	Create Reliance demo flow	March 20
Priya	Update CTO on timeline risks	Immediate

Final Assessment

Overall Project Health: 7.5/10

Category	Score
Model Performance	8/10
Data Pipeline	9/10
Reliability	6/10
Scalability	8/10
Operational Readiness	7/10
Launch Readiness	6.5/10

Top Priorities

1. Fix table extraction bug.
2. Improve contract extraction accuracy.

3. Resolve data residency constraints.
4. Benchmark Azure OCR.
5. Secure additional GPU capacity.
6. Validate V1 launch feasibility after March 27 benchmark results.

Verdict

The platform is in a strong position for invoices and purchase orders, but **contract extraction remains the single largest technical and business risk**. The success of the April 18 launch depends largely on resolving this area within the next two weeks.